ENGINEERING REBOOT 2026

Day 32 Field Manual

Version 1.0 – Implementation Edition

---

1. Mission

Welcome to **Day 32** of the Engineering Reboot 2026 program.

Yesterday you learned how to define classes, create objects, use constructors, and instantiate multiple independent objects. Today, you move beyond object creation and begin designing objects that manage their own behavior.

The objective is to understand that an object is more than a container of data—it is responsible for operating on that data while protecting its own internal state.

By the end of today, you should be able to design classes whose public methods form a clean interface while the object's internal data remains under its control.

---

2. Engineering Context

Professional software systems are built from collaborating objects.

Instead of allowing every part of a program to modify data directly, object-oriented software exposes carefully designed methods that enforce business rules.

For example:

Instead of

```
account.balance -= 500
```

professional software prefers

```
account.withdraw(500)
```

because the object itself determines whether the withdrawal is valid.

This principle is known as **encapsulation**.

Today's work begins developing the engineering habit of allowing objects—not outside code—to manage their own state.

---

3. Learning Objectives

Technical Objectives

Upon completion of Day 32, you should be able to:

- Write meaningful instance methods.
- Modify object state through methods.
- Return information from methods.
- Distinguish between reading and modifying object data.
- Apply basic encapsulation principles.
- Understand Python naming conventions for protected members.

Engineering Objectives

Develop the habit of:

- Giving objects responsibility for their own behavior.
- Designing clean public interfaces.
- Avoiding unnecessary direct attribute modification.
- Writing methods that perform one well-defined task.
- Keeping business logic inside the class whenever practical.

---

4. Core Concepts

Today's lesson introduces the following concepts:

- Instance methods
- Public methods
- Internal state
- Encapsulation
- Protected attributes (`_attribute`)

- Private attributes (`__attribute`)
- Accessor (getter) methods
- Mutator (setter) methods
- Business rule validation inside methods

---

5. Exercises

Exercise 1 — Building Useful Methods

Filename

`employee_methods.py`

Objective

Extend yesterday's Employee class by adding meaningful behaviors.

Requirements

Create an Employee class containing:

- employee_id
- name
- department
- salary

Implement methods that:

- Display employee information.
- Return annual salary.
- Apply a percentage raise.
- Transfer an employee to another department.

Create multiple Employee objects and demonstrate each method.

Exercise 2 — Understanding Encapsulation

Filename

`bank_account.py`

Objective

Introduce encapsulation through controlled access.

Requirements

Create a BankAccount class.

Attributes:

- account_number
- owner
- _balance

Methods:

- deposit()
- withdraw()
- get_balance()

Validation requirements:

- Reject negative deposits.
- Reject withdrawals greater than the current balance.
- Display informative messages for invalid operations.

Exercise 3 — Private Members

Filename

`encapsulation_demo.py`

Objective

Understand Python's private attribute naming convention.

Requirements

Create a simple class using:

`__balance`

Experiment with:

- Accessing inside the class.
- Attempting access outside the class.
- Using a getter method.

Observe Python's name mangling behavior.

Exercise 4 — Object Collaboration

Filename

`employee_payroll_demo.py`

Objective

Practice invoking multiple methods across several objects.

Requirements

Create several Employee objects.

Demonstrate:

- Raises
- Department transfers
- Annual salary calculations
- Updated employee reports

---

6. Mini Project

Banking System

Objective

Design a simple banking system demonstrating encapsulation.

BankAccount Class

Attributes

- account number
- account owner
- current balance

Methods

- deposit()
- withdraw()
- get_balance()
- display_account()

Business Rules

- Deposit amount must be positive.
- Withdrawal amount must be positive.
- Withdrawal cannot exceed current balance.
- Balance must only be modified through class methods.

Main Program

Create multiple bank accounts.

Demonstrate:

- successful deposits
- successful withdrawals
- rejected withdrawals
- rejected deposits
- account summaries
- final balances

Expected Learning Outcome

By completing this project you should understand why object behavior belongs inside the class rather than being scattered throughout the application.

---

7. Validation Matrix

Before completing Day 32, verify that you can answer **Yes** to each item.

Requirement	Complete
Created classes with multiple methods	☐
Modified object state through methods	☐
Returned values from methods	☐
Applied encapsulation principles	☐
Used protected/private attributes appropriately	☐
Completed the banking mini-project	☐
Tested both valid and invalid operations	☐

---

8. Evidence Checklist

Capture evidence for:

- Source code for every exercise
- Program execution for every exercise
- Banking mini-project source code
- Banking mini-project execution
- Git operations
- Repository status after completion

---

9. Engineering Review

At the end of Day 32, evaluate your implementation.

Consider the following questions:

- Does each method have a single responsibility?
- Does the class protect its own internal state?
- Is validation performed inside the class rather than by the caller?
- Are methods clearly named and easy to understand?
- Would another programmer immediately understand the public interface?

---

10. Notes.md Guidance

Document:

- Your understanding of encapsulation.
- Differences between public, protected, and private attributes.
- When methods should return values versus modifying object state.
- Observations about Python's implementation of private members.

---

11. Journal.md Guidance

Reflect on:

- How your perspective on object-oriented programming changed after implementing encapsulation.
- Whether designing object behavior felt more natural than procedural programming.
- Challenges encountered while moving business logic into the class.
- Lessons learned that you will apply in future object-oriented designs.

---

12. Completion Checklist

Day 32 is complete when you have:

- Completed all exercises.
- Completed the Banking System mini-project.
- Verified all validation requirements.
- Captured execution evidence.
- Updated `Notes.md`.
- Updated `Journal.md`.
- Committed changes to Git.
- Pushed changes to the remote repository.
- Verified repository status.

---

13. Preview – Day 33

Tomorrow you will continue expanding your object-oriented design skills by introducing **relationships between objects** through a **Library Management System**, where multiple class instances interact to model a larger software system. This progression prepares you for increasingly realistic software architecture in the remainder of Week 7.